

CudaForge: An Agent Framework with Hardware Feedback for CUDA Kernel Optimization

Zijian Zhang* Rong Wang* Shiyang Li Yuebo Luo

Mingyi Hong[†] Caiwen Ding[†]

University of Minnesota, Twin Cities

{zha00175, wan00559, li004074, luo00466, mhong, dingc}@umn.edu

Abstract

Developing efficient CUDA kernels is increasingly critical for AI applications such as large-scale LLM training. However, manual kernel design is both costly and time-consuming, motivating automatic approaches that leverage LLMs for code generation. Existing methods for automatic kernel generation, however, often produce low-efficiency kernels, incur high computational overhead, and fail to generalize across different settings.

In this work, we propose **CudaForge**, a training-free multi-agent workflow for CUDA kernel generation and optimization. Our workflow is inspired by the iterative workflow of human experts, which contains steps such as developing initial kernels, testing correctness, analyzing hardware feedback, and iterative improvement. More specifically, **CudaForge** employs two LLM agents: a Coder and a Judge, that iteratively generate, correct, and optimize CUDA kernels, while integrating hardware feedback such as Nsight Compute (NCU) metrics. In our extensive evaluations, we show that **CudaForge**, by leveraging base models like OpenAI-o3, achieves 97.6% correctness of generated kernels and an average $1.68\times$ speedup over PyTorch baselines, substantially surpassing state-of-the-art models including OpenAI-o3 and Kevin on KernelBench, while further scaling up maximum iteration rounds increases **CudaForge**'s performance to $2.27\times$ speedup, showing its strong capability in practice. Beyond accuracy and speed, **CudaForge** demonstrates strong generalization across GPUs (A100, RTX 6000, 4090, 3090) and base models (OpenAI-o3, GPT-5, gpt-oss-120B, Claude-Sonnet-4, QwQ-32B), while maintaining high efficiency. In particular, generating an optimized kernel takes about 26.5 minutes on one RTX6000 and incurs about \$ 0.3 API cost, which is significantly cheaper than existing agentic work that costs 6 H100 hours and \$ 5 API cost per kernel. Our results highlight that multi-agent, training-free workflows can enable cost-effective, generalizable, and high-performance CUDA kernel optimization. Code available at <https://github.com/OptimAI-Lab/CudaForge>

1 Introduction

Motivation. CUDA has become the *de facto* standard for deep learning training because modern frameworks such as PyTorch and TensorFlow are deeply integrated with NVIDIA's optimized GPU libraries [?]. Efficient CUDA kernels are crucial for accelerating deep learning workloads [? ?] .

However, developing high-efficiency CUDA kernels is known to be challenging with a steep learning curve, requiring deep expertise in GPU architectures and parallel programming [?]. For

*Co-first author. Equal contribution; order decided by a coin toss.

[†]Corresponding author.

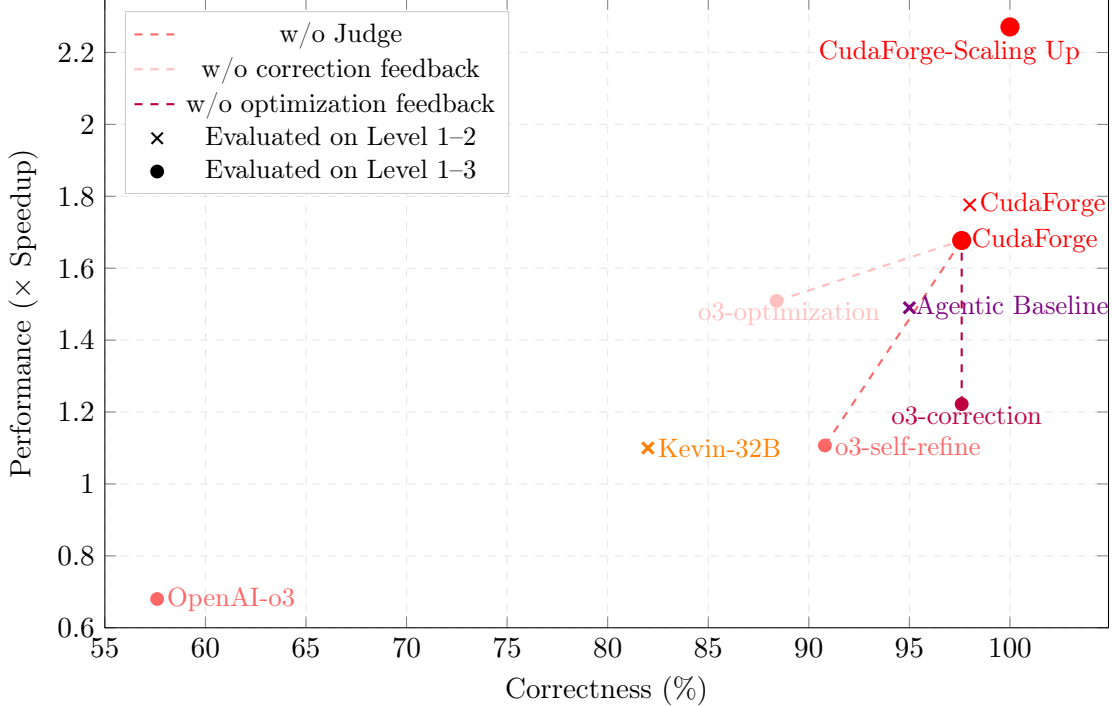


Figure 1: CudaForge achieves state-of-the-art results on KernelBench in both correctness and performance, surpassing RL-based methods such as Kevin-32B [?], the agentic baseline [?], and OpenAI-o3 [?]. To further evaluate the effectiveness of our design, we additionally develop three customized variants of OpenAI-o3: o3-self-refine, o3-correction, and o3-optimization, which serve as baselines for ablation comparison. Scaling up maximum iteration rounds(CudaForge-Scaling Up) further improves CudaForge’s performance to $2.27\times$ speedup. Experimental details are provided in Section 3.

example, it took more than 2 years from the debut of the Hopper GPU architecture to the release of FlashAttentionV3 [?], which is specially designed for Hopper GPUs.

This high development barrier has driven growing interest in finding automated ways of generating highly efficient and customized CUDA kernels. For example, some work [?] [?] employs auto-tuning and evolutionary search to automatically explore kernel implementation spaces and optimize low-level parameters for specific hardware. More recently, there has been a growing interest in leveraging large language models (LLMs) to perform such tasks. LLM is believed to hold great promise in generating efficient and high-quality kernels, due to its capability of code generation in other domains, such as Python and C++ [? ?].

Existing Works and Key Challenges. Generally, using LLMs for CUDA kernel generation is still in an early stage. In KernelBench [?], the authors attempt to directly use state-of-the-art (SOTA) models, such as OpenAI-o1 and Claude-3.5-Sonnet, to generate kernels. However, it has been observed that these SOTA models still struggle to produce correct or performant kernels out of the box, revealing fundamental limitations of existing LLMs in this domain.

To address this gap, recent studies have explored two main paradigms. The first approach is based on reinforcement learning (RL) [? ?]. CUDA-L1 [?] and Kevin [?] adopt RL to enhance LLMs’ ability to generate correct and performant CUDA code.

The second approach is based on AI agents. In particular, in an independent and contemporaneous

work [?] ¹, researchers have explored agentic frameworks at inference time. Agents project PyTorch methods into CUDA kernel designs, then the CUDA kernels are further refined by sampling new kernels and verification filtering. This design effectively improves correctness in CUDA kernel generation without the high cost of RL training.

Despite these advances, several key challenges remain:

(C1) Limited kernel efficiency. While RL-based methods improve LLMs’ ability to generate CUDA kernels, their optimization capability remains insufficient. For example, the kernels generated by Kevin-32B only achieve an average speedup of $1.10\times$ over KernelBench Level 1-2, even after sampling 16 parallel trajectories with 8 refinement turns for each kernel [?]. As another example, CUDA-L1 often fails to directly optimize the CUDA kernels, but produces official implementations of PyTorch [?] (see Appendix C for details).

(C2) High training and inference cost. RL-based approaches such as [?] require substantial computational resources and long training cycles, making them unsuitable for low-resource or rapid-prototyping settings. In addition, multi-stage agentic pipeline developed by [?] incurs high inference costs (about 6 H100 hours and \$5 API cost per kernel), which greatly limits its practical applicability of the approach.

(C3) Lack of hardware feedback. Human experts typically follow an iterative workflow to develop CUDA kernels through testing and refinement. They rely on hardware feedback like Nsight Compute (NCU) ² to identify bottlenecks and optimize kernels accordingly [?]. In contrast, RL-based approaches [?] train LLMs to directly generate or optimize kernels, but do not leverage hardware feedback at all. As a result, they rely on blind exploration during generation, lacking targeted guidance. This often leads to suboptimal kernel efficiency, limiting their practical applicability.

These challenges raise a natural question: *Can we design a simple but effective hardware-aware approach that reliably produces efficient CUDA kernels at low cost?*

Our Contributions. To address these challenges, we propose **CudaForge**, a **simple, effective, and low-cost** multi-agent workflow for CUDA kernel generation and optimization, as shown in Figure 2. Our workflow is inspired by the iterative workflow of human experts [?], which contains steps such as developing initial kernels, testing correctness, analyzing hardware feedback, and iterative improvement.

This workflow involves two specialized LLM agents that iteratively generate and optimize CUDA kernels: a **Coder**, which generates kernels given task instructions and Judge feedback, and a **Judge**, which analyzes kernels and hardware feedback to guide the Coder generation. One key novelty of **CudaForge** is its integration of external hardware feedback, including GPU specifications and Nsight Compute (NCU) metrics, enabling the Judge to identify performance bottlenecks like human experts and provide targeted optimization guidance to the Coder.

Compared to single-LLM approaches that generate and evaluate code using the same LLM, our framework separates these roles into an *independent* Coder and Judge, enabling more specialized reasoning and more reliable iterative refinement. Unlike RL-based methods, **CudaForge** is training-free, avoiding the substantial cost of policy training. It is also hardware-aware, allowing it to tailor CUDA kernel optimizations to the underlying system, making the proposed framework easily generalizable across different GPUs. Finally, in contrast to existing multi-agent frameworks [?], **CudaForge** is lightweight and cost-efficient, running in just 26.5 minutes on a single RTX 6000 GPU and about \$ 0.3 per kernel in API on average, while still achieving significantly better performance.

We evaluate **CudaForge** on 250 KernelBench tasks from Level 1 to Level 3. Though these tasks are challenging, **CudaForge** attains a 97.6% correctness rate and delivers an average speedup of $1.68\times$

¹published on arxiv Sept 16th, 2025

²Nsight Compute (NCU) is NVIDIA’s official kernel-level profiler for CUDA programs.

over PyTorch baselines, which significantly outperforms advanced RL model like Kevin-32B, advanced frontier model like OpenAI-o3 [?] and Agentic Baseline [?], shown in Figure 1. Further, we have conducted comprehensive ablation studies of the features of CudaForge, such as its effectiveness across multiple GPU architectures, its inference-time scalability by increasing the number of generation, and the effect of different base models. Overall, we observed that the proposed CudaForge achieves robust performance in all these settings.

These findings highlight the key contribution of this work: The proposed LLM agent workflow CudaForge is simple but effective: at very low cost, it develops performant CUDA kernels for many practical tasks, for a variety of GPU architectures and base models. It also exhibits strong test-time scaling capabilities where solution quality can improve substantially while increasing its iteration rounds. These results demonstrate CudaForge’s strong practical applicability.

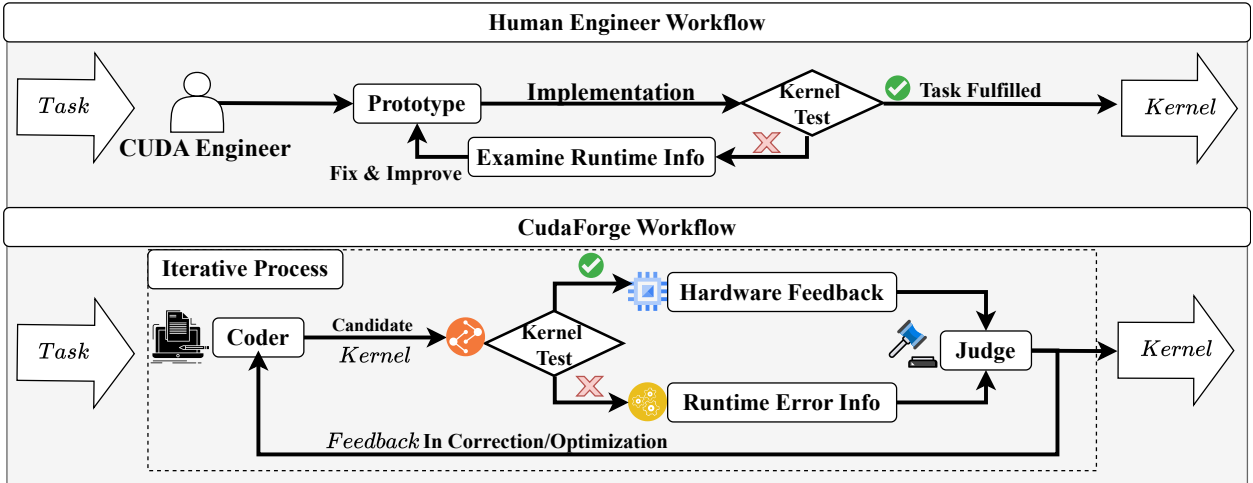


Figure 2: Comparison between human and CudaForge workflows. Top: Human experts iteratively refine kernels by writing a prototype, testing it, and analyzing runtime feedback. Bottom: CudaForge mimics human workflow with two specialized agents (Coder and Judge). The Coder generates candidate kernels, while the Judge analyzes runtime info and hardware feedback to provide correction or optimization feedback. The process iterates until it reaches maximum round N .

2 The CudaForge Framework for CUDA Kernel Optimization

2.1 CudaForge Framework

Given a CUDA kernel generation task, the objective is to generate a kernel that is functionally equivalent to its PyTorch reference while achieving the lowest possible execution latency.

Inspired by the iterative workflow of human experts [? ? ?], we design CudaForge as an iterative multi-agent framework, illustrated in Figure 2. The framework involves two independent agents: a **Coder** and a **Judge**. The Coder generates candidate kernels based on the task description and feedback from the Judge, while the Judge evaluates each candidate using the kernel itself, hardware feedback, and runtime information.

More specifically, given a CUDA kernel generation task, the Coder first receives the task requirements and PyTorch reference implementation, then produces an initial candidate kernel. This kernel is compiled and executed on test cases to check its correctness. If it fails, the Judge inspects *runtime information* (e.g. compilation errors, mismatched outputs with the PyTorch reference) and analyzes the faulty kernel. It then returns correction feedback (e.g. missing header file) to guide the next round. Once a kernel candidate passes the correctness test, the Judge profiles it with the NCU

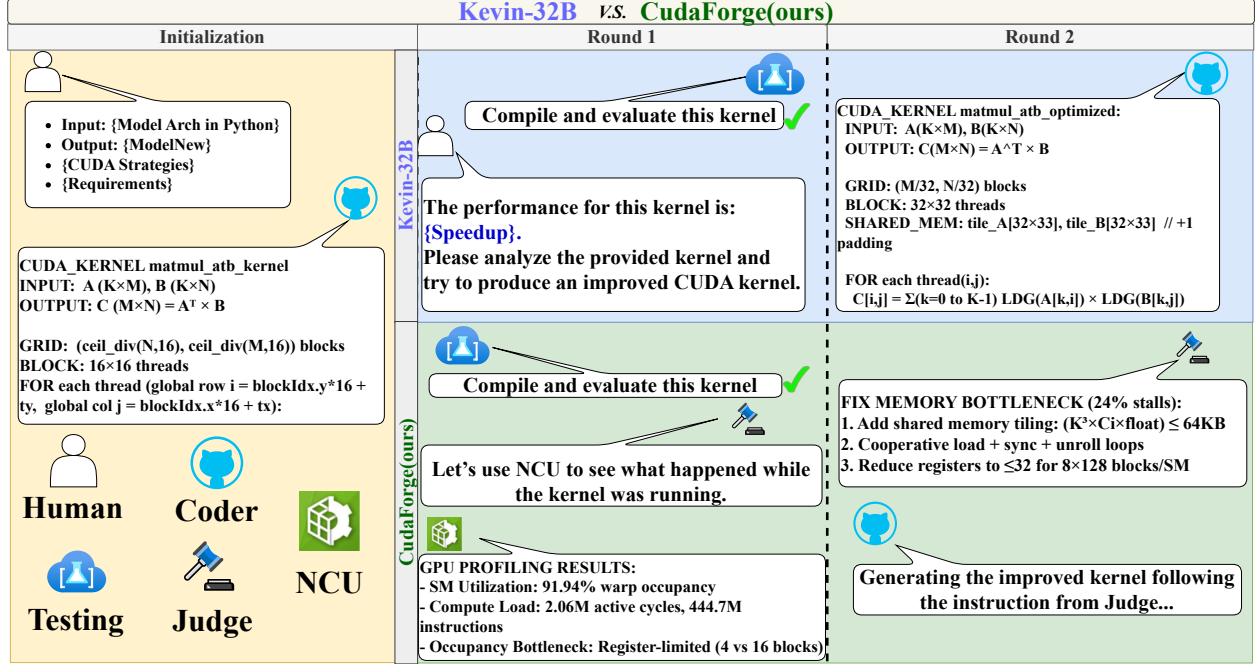


Figure 3: The overview of how CudaForge optimizes kernels, compared with Kevin-32B. **Top:** the pipeline of the RL-based Kevin-32B, which relies solely on textual refinement and thus performs blind exploration. **Bottom:** our CudaForge workflow, which leverages hardware feedback to guide kernel optimization. When the Coder in CudaForge generates a correct candidate kernel in Round 1, the system profiles it using Nsight Compute (NCU) to obtain NCU metrics. In Round 2, the Judge analyzes these metrics and GPU specifications to identify performance bottlenecks (e.g., register- or memory-limited) and provides targeted optimization feedback. The Coder then refines the kernel accordingly. Compared with Kevin-32B, which only refines based on speedup scores, our framework achieves more interpretable and effective performance improvements through hardware-aware iteration.

tool to obtain NCU metrics (e.g. memory throughput, occupancy, warp efficiency). Together with GPU specifications, these metrics form the *hardware feedback* that allows the Judge to identify the dominant bottleneck (e.g. compute-bound or memory-bound) and provide one specific optimization feedback (e.g. using shared memory) to the Coder.

In the next round, the Coder is prompted with the previous kernel, Judge feedback, and the original task requirements, and generates a corrected or optimized kernel. This process repeats for up to N rounds, after which we select the most efficient correct kernel as the final solution.

CudaForge achieves reliability and efficiency through **three key design choices**. First, it adopts a two-agent system where the Coder focuses on generation and the Judge on evaluation, separating the “cognitive” load (See Section 3.6 for ablation study). The Coder receives only feedback from the Judge, while the Judge uses hardware and runtime information to guide generation and optimization. This division of labor mirrors human workflows and mitigates the risk of overlooking errors or inefficiencies. Second, the framework follows an iterative optimization process, progressively correcting errors and improving efficiency across rounds. This enables stable refinement, especially on hard tasks. Third, it explicitly incorporates hardware feedback, such as GPU specifications and NCU metrics, so that the Judge can pinpoint bottlenecks and provide actionable guidance to the Coder. This targeted optimization avoids blind exploration and ensures directed performance gains.

2.2 Component Design

Design of Coder. The Coder acts as the generative module in our framework, responsible for producing functionally correct and efficient CUDA kernel candidates given the task description, previous candidates, and feedback from the Judge. Directly generating a CUDA kernel from scratch is a highly challenging task for LLMs [?]. Following the setting of KernelBench [?], we provide the Coder with a one-shot demonstration in the prompt, which includes an example pair of a PyTorch reference and its corresponding CUDA kernel implementation. This example serves as a structural and stylistic guide, helping the Coder understand the expected syntax, memory access patterns, and API usage required for valid kernel generation.

Moreover, we find that managing the Coder’s memory scope is crucial for stable and efficient generation. Prompting the model with the entire dialogue history introduces excessive context redundancy, often leading to hallucinated kernel code and higher API cost. To mitigate these issues, we adopt a lightweight memory design, where the Coder is prompted round by round without retaining the full conversation history. In each round, it only receives the *latest* feedback from the Judge, previous kernel candidate, and task description, enabling it to focus on refining or correcting the previous version. This design greatly improves code stability, reduces inference cost, and allows the model to perform precise, hardware-driven optimizations. See Appendix A for Coder’s prompt.

Design of Correctness Tests. After the Coder generates a candidate kernel, our workflow proceeds to verify its *correctness* through a two-stage testing process, including compilation and execution. The compilation stage ensures that the generated kernel is syntactically valid and can be successfully compiled into executable CUDA code. The execution stage then evaluates the functional correctness of the compiled kernel by running it on predefined test cases. We compare the kernel’s outputs with those from the PyTorch reference implementation under the same inputs, and consider the kernel functionally correct only if the numerical difference between the two outputs is within a small tolerance (set to $1e-4$ in our experiments). A kernel is regarded as correct only when it passes both compilation and execution stages successfully, which is a commonly adopted criterion [? ? ?].

Design of Judge. The Judge serves as the evaluation and guidance module in our framework. Its primary responsibility is to provide actionable feedback to guide the Coder’s next round given task description, current candidate kernel, and hardware feedback or runtime information. The

Judge operates in two distinct modes depending on kernel validity: If the kernel fails compilation or produces incorrect outputs, the Judge performs the *correction mode*, identifying issues such as invalid indexing, race conditions, or missing headers, and returns **correction feedback** that instructs the Coder to fix them. If the kernel passes all correctness checks, the Judge enters *optimization mode*, where it uses hardware feedback to identify the dominant bottleneck—e.g., memory-bound, compute-bound, or occupancy-limited—and formulates **optimization feedback** accordingly. Please see Section 2.3 for details of using hardware feedback.

Similar to the design of the Coder, the Judge does not retain the full conversation. In each round, the Judge is prompted with the relevant mode and acts according to the corresponding role. Finally, it generates structured feedback in JSON format, which is then extracted and passed to the Coder for the next round. This design ensures modularity and clarity in the interaction between agents, allowing the Judge to efficiently translate hardware-level profiling signals into precise, interpretable feedback that drives iterative kernel optimization. See Appendix A for the Judge’s prompt.

2.3 How to Integrate Hardware Feedback

In this subsection, we describe in detail a key design consideration, which enables CudaForge to utilize hardware feedback for kernel performance optimization. The hardware feedback integrates static GPU specifications (e.g. architecture, memory bandwidth, per-thread register limits, per-SM shared-memory capacity) with performance metrics (e.g. memory throughput, occupancy, and warp efficiency) from Nsight Compute (NCU) collected during kernel execution. By cross-referencing GPU specifications and NCU metrics, the Judge infers the kernel’s primary performance-limiting cause and bottleneck. Figure 3 illustrates how Judge uses hardware feedback to optimize kernels.

Just as CUDA engineers focus on key indicators, we choose not to pass the entire set of NCU metrics to the Judge. Feeding all metrics can overwhelm the decision process with excessive, partially redundant signals and lead to unstable judgments (See Section 3.6 and Appendix B.1 for ablation study and case study). Instead, we design a novel protocol which profiles a subset of critical metrics provided by NCU and forward them to Judge so that we can improve the quality of the judge outputs. More specifically, the key subset of metrics are selected off-line (before the workflow starts to work), through the following steps:

(Step 1) Kernel sampling and Selection: We first profile key metrics on some preselected representative tasks (e.g., Conv2D, MatMul) to prepare a reliable metric set. Specifically, for each task we run 100 self-refine (repeating the cycle generating $\rightarrow$ execute/profile $\rightarrow$ evaluate $\rightarrow$ repair/optimize) with a single SOTA model (e.g. OpenAI-o3), collect the generated and correct kernels, and select 10 with the largest speed disparity (fastest vs. slowest). See Algorithm 1.

Algorithm 1: Step 1: Kernel Sampling and Selection

Input: Task set $Task = \{T_1, T_2, \dots, T_n\}$

Output: Selected subsets K_i^* for each task T_i

for $i \leftarrow 1$ **to** n **do**

$K_i \leftarrow \emptyset$;

for $j \leftarrow 1$ **to** 100 **do**

$k_j \leftarrow \text{generate_kernel}(T_i)$;

$K_i \leftarrow K_i \cup \{k_j\}$;

end

 Sort K_i in nondecreasing order according to kernel runtime;

$m \leftarrow |K_i|$;

// Here $m = 100$

$K_i^* \leftarrow \{K_i[1], K_i[2], K_i[3], K_i[4], K_i[5], K_i[m-4], K_i[m-3], K_i[m-2], K_i[m-1], K_i[m]\}$;

end

memory-bound when memory throughput is high but computing resource utilization is low, and then it will choose memory related metrics as critical metrics in this round. After this, Judge will generate suggestions on how to modify the kernel to address the current critical bottleneck. The Coder incorporates this guidance in the next round generation accordingly. This mechanism enables our multi-agent system focus on addressing only one critical program bottleneck in each round, and eventually optimizes overall kernel performance step by step in iterative rounds, just like human experts’ real workflow.

3 Experiments

3.1 Benchmark and Evaluation

We evaluate our method and baselines on **KernelBench** [?], a popular benchmark designed to assess the ability of LLMs to generate CUDA kernels. KernelBench consists of multiple difficulty levels, and we adopt all tasks from Level 1 to Level 3, resulting in a total of 250 tasks. Specifically, Level 1 contains relatively simple 100 tasks involving basic operators (e.g., matrix multiplication), Level 2 includes medium-difficulty 100 tasks composed of multi-step operator combinations, and Level 3 contains 50 challenging tasks involving full neural network architectures (e.g., AlexNet). Each task is accompanied by a reference PyTorch implementation and predefined input/output specifications, which enables fully automated and reliable evaluation of both correctness and performance. Details of KernelBench are provided in Appendix D.1.

Due to the high computational cost of running experiments for some ablation studies, we do not evaluate them on the entire KernelBench benchmark. Instead, we construct a stratified random subset of tasks, denoted as $\mathcal{D}^*$, by sampling proportionally from each difficulty level of KernelBench. This ensures that $\mathcal{D}^*$ maintains the same task distribution as the full benchmark while enabling fair and efficient evaluation. Specifically, $\mathcal{D}^*$ contains a total of 25 tasks, with 10 tasks in Level 1, 10 tasks in Level 2 and 5 tasks in Level 3. More details on the construction of $\mathcal{D}^*$ are provided in Appendix D.2.

We evaluate model performance on KernelBench using the following metrics: (1) **Correctness**: the fraction of tasks for which the generated kernel compiles successfully and produces outputs identical to the PyTorch reference on all test cases. (2) **Performance**: the ratio of the execution speed (tested on a specific GPU), between a correct generated kernel and its PyTorch reference. (3) **Fast₁**: the proportion of correct kernels whose execution speed exceeds their PyTorch reference. (4) **Median speedup**: the median of ‘Performance’ values across all tasks, reflecting typical rather than average behavior. (5) **75th percentile speedup**: the 75th percentile of Performance values, capturing upper-quartile efficiency.

For methods that perform iterative refinement or generate multiple candidates (including CudaForge), we report the best-performing correct kernel among all candidates for each task.

3.2 Settings & Baselines

In our main results, we instantiate CudaForge with OpenAI-o3 as both the Coder and the Judge as our *default* setting. We set the maximum number of iteration rounds to $N=10$ to balance performance improvements and inference cost. Unless otherwise stated, all methods are evaluated under the same compilation/runtime environment in Quadro RTX 6000 and task-specific test suites.

To contextualize the performance of CudaForge and assess the effect of advanced foundation models, we include the following baselines for the main results and ablation studies:

- OpenAI-o3: Using OpenAI-o3 for one-shot generation without iteration;

Table 1: Main results on KernelBench (Level 1-3, 250 tasks). Results of Agentic Baseline is on Level 1 and 2. All experiments here are run in RTX 6000. Methods evaluated on $\mathcal{D}^*$ are marked with *. CudaForge-Scaling Up means scaling up maximum iteration rounds

Method	Correct $\uparrow$	Median $\uparrow$	75% $\uparrow$	Perf $\uparrow$	Fast ₁ $\uparrow$
OpenAI-o3	57.6%	0.390	1.014	0.680	31.60%
o3-self-refine	90.8%	1.012	1.209	1.107	55.20%
o3-correction	97.6%	1.031	1.238	1.222	59.60%
o3-optimization	88.4%	1.061	1.483	1.509	64.00%
Agentic Baseline(Level 1 & 2)	95.0%	—	—	1.490	—
CudaForge(full metrics)*	100%	1.280	1.489	1.414	80.00%
CudaForge	97.6%	1.107	1.592	1.677	70.80%
CudaForge(Level 1 & 2)	98.0%	1.112	1.617	1.776	71.50%
CudaForge*	100%	1.322	1.736	1.767	84.00%
CudaForge-Scaling Up*	100%	1.317	1.777	2.265	92.00%

Table 2: Main results on KernelBench (Level 1-3, 250 tasks) of CudaForge in RTX 6000.

Task	Correct $\uparrow$	Median $\uparrow$	75% $\uparrow$	Perf $\uparrow$	Fast ₁ $\uparrow$
Level 1	96%	1.044	1.751	1.448	54.0%
Level 2	100%	1.124	1.427	2.104	89.0%
Level 3	96%	1.081	1.510	1.283	68.0%

- o3-self-refine(our baseline): Using OpenAI-o3 for ten rounds of self-refinement without a Judge, where the model relies solely on itself to correct and optimize kernels given hardware feedback;
- o3-correction(our baseline): A variant of CudaForge where the Judge provides only correctness feedback but no optimization feedback;
- o3-optimization(our baseline): A variant of CudaForge where the Judge provides only optimization feedback but no correction feedback;
- Kevin-32B: a strong RL-based model for CUDA kernel optimization from [?]. We directly take results from their official paper.
- Agentic Baseline: the agentic workflow from [?], a strong multi-agent baseline, which uses an LLM ensemble including both reasoning (o3 & o4-mini) and conventional LLMs (Claude Sonnet 3.7 & GPT-4.1).
- CudaForge (full metrics): a variant of CudaForge where the Judge leverages the entire set of NCU metrics.

This baseline setting enables a comprehensive comparison across (i) base model vs. corresponding agent-based method, (ii) the presence/absence of Judge feedback, (iii) RL-based vs. training-free agent-based approaches, and (iv) different agentic methods.

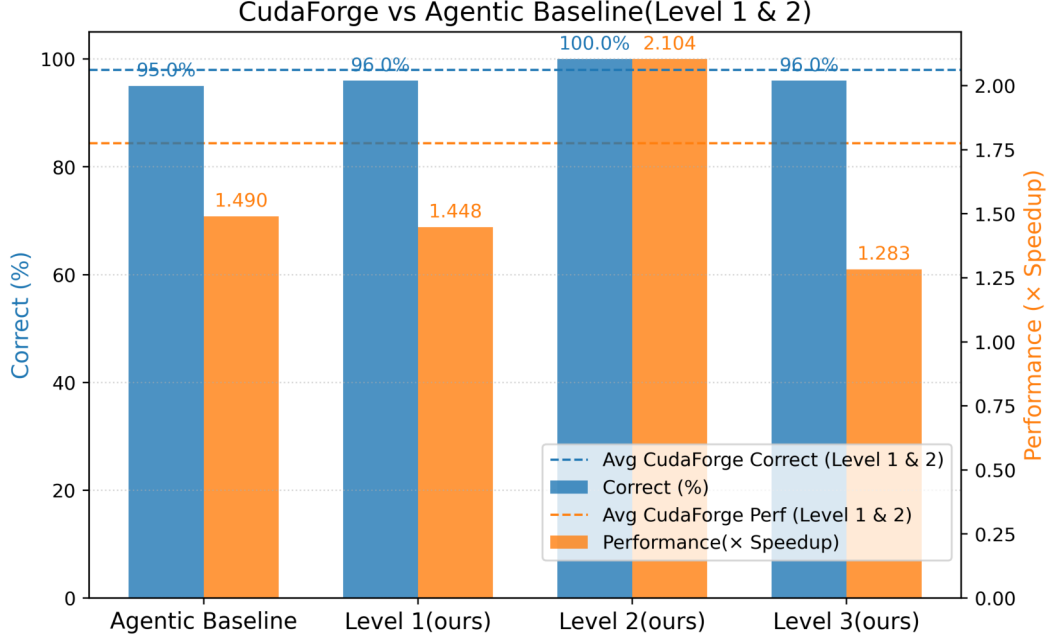


Figure 4: Comparison of correctness and performance between CudaForge and the Agentic Baseline on KernelBench. Dashed lines denote average results of CudaForge over Level 1 and 2. CudaForge outperforms Agentic Baseline on KernelBench Level 1 and 2, and it also achieves strong performance in Level 3.

3.3 Main Results in RTX 6000

Table 1 reports the main results in KernelBench. CudaForge consistently outperforms all baselines across all metrics, both in the entire KernelBench and in the stratified subset $\mathcal{D}^*$.

On KernelBench, CudaForge attains **97.6%** correctness with an average performance of **1.677×**, and **70.8% Fast₁**, while achieving a median speedup of 1.107× with a 75th percentile speedup of 1.592×. These results significantly improve over the base model OpenAI-o3 and other ablated variants, including o3-self-refine, o3-correction and o3-optimization.

On the reduced dataset $\mathcal{D}^*$, CudaForge achieves 100% correctness, a median speedup of 1.322×, a 75th percentile speedup of 1.736×, an average performance of 1.767×, and 84.0% Fast₁. This substantially surpasses CudaForge(full metrics), which reaches only 1.280× median, 1.489× at the 75th percentile, 1.414× performance, and 80% Fast₁. Moreover, after scaling up maximum iteration rounds, CudaForge-Scaling Up achieves a stronger performance, with **2.265×** speedup. We will further discuss ablation studies using $\mathcal{D}^*$ in Section 3.6 and scaling up maximum iteration rounds in Section 3.7.

We also compare CudaForge with Agentic Baseline³ on KernelBench Level 1 and Level 2. As shown in Table 1, 2 and Fig 4, CudaForge achieves 98% correctness and an average speedup of 1.776×, which outperforms Agentic Baseline (95.0%, 1.490×), especially in speedup. This result shows our advantage compared to existing agentic work.

Notably, on Level 3, which represents the most challenging level of KernelBench, CudaForge achieves **96%** correctness and an average **1.283×** speedup. Given the complexity of Level 3 tasks, which

³Note that it only reports results in Level 1 and 2, and we directly take the results from their paper since the paper has not open-sourced the code.

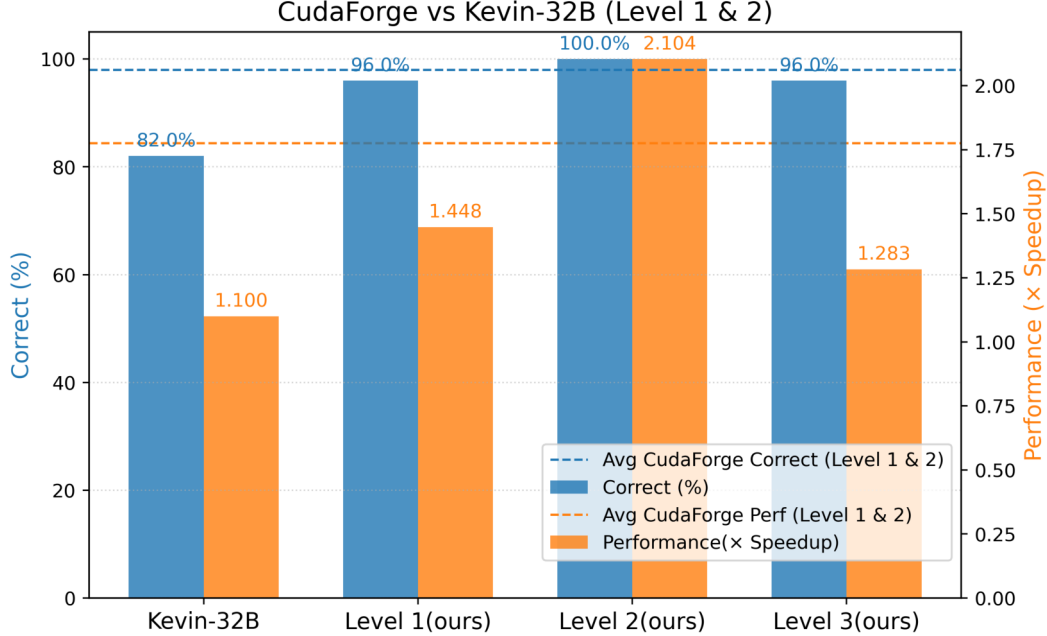


Figure 5: Comparison of correctness and performance between CudaForge and Kevin-32B on KernelBench. Dashed lines denote average results of CudaForge over Level 1 and 2. While training-free, CudaForge outperforms Kevin-32B in KernelBench Level 1-2, and gets outstanding results in Level 3.

involve full neural network architectures and multi-stage operations, these results demonstrate that CudaForge is capable of reliably generating and optimizing highly complex CUDA kernels, where prior approaches [? ?] have not explored it.

3.4 Comparison with Kevin-32B on H200

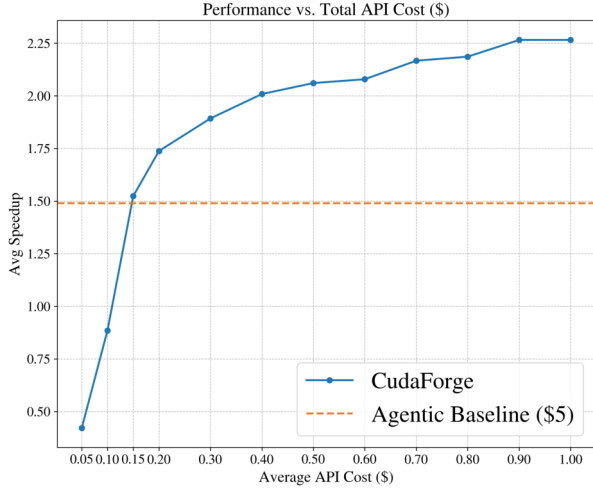
In this subsection, we compare CudaForge with Kevin-32B [?]. However, since Kevin-32B is not open-sourced—meaning that key details such as test-time prompts, evaluation setup, and benchmark specifications are not publicly available—a fully fair and reproducible comparison cannot be conducted. According to the original paper, Kevin-32B was trained and evaluated on H200 GPUs using a self-constructed benchmark of comparable difficulty to KernelBench Levels 1 and 2. To ensure the comparison is as fair as possible under these constraints, we re-evaluate CudaForge on the same H200 hardware across KernelBench Levels 1–3, aligning as much as possible our computational environment and evaluation protocol with those reported for Kevin-32B.

As shown in Figure 5, CudaForge achieves consistently higher correctness and performance across all levels. In the tasks comparable to Kevin’s benchmark (Levels 1 & 2), CudaForge reaches an average of **98.0%** correctness and **1.662×** performance, outperforming Kevin-32B’s 82.0% correctness and 1.10× speedup. Even in the most challenging Level 3 tasks, CudaForge maintains strong results with **96.0%** correctness and **1.261×** performance, demonstrating its robustness on complex CUDA kernels.

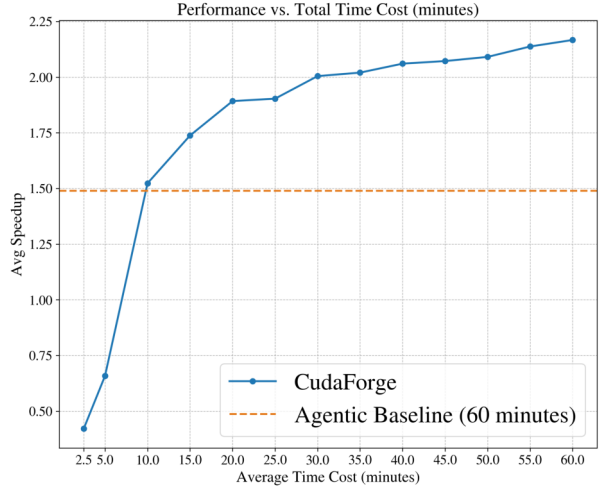
These findings highlight that, despite being a training-free framework, CudaForge surpasses the RL-trained Kevin-32B in both reliability and efficiency. We attribute this advantage to the effective use of hardware feedback and the design of workflow, which enables CudaForge to perform targeted

Table 3: Comparison of API and computation time cost between CudaForge and the Agentic Baseline [?].

Method	Metric	Average	Level 1	Level 2	Level 3
Agentic Baseline	API Cost (\$)	5.0	—	—	—
	Time (min)	60.0	—	—	—
CudaForge	API Cost (\$)	0.30	0.29	0.30	0.33
	Time (min)	26.5	28.5	24.1	27.1



(a) Performance vs. API cost of CudaForge.



(b) Performance vs. computation time of CudaForge.

Figure 6: Relationship between cost and performance of CudaForge. Both API and computation time exhibit a monotonic correlation with performance. CudaForge already surpasses the Agentic baseline while using no more than \$0.15 and 10 minutes per task.

hardware-aware optimizations rather than rely solely on (potentially inefficient) RL training.

3.5 API and Computation Time Cost of CudaForge

We evaluate both the API and computation time costs of CudaForge on the KernelBench dataset $\mathcal{D}^*$. The API cost is measured as the total expenditure per kernel generation task, while the computation time is measured as the end-to-end wall-clock time, including kernel compilation and execution, model generation, and NCU profiling. All experiments are conducted on a single RTX 6000 GPU.

As shown in Table 3, CudaForge requires on average only 26.5 minutes of wall-clock time on a single RTX 6000 GPU and incurs merely \$0.3 of API cost per kernel. This is highly cost-efficient compared with the Agentic Baseline [?], which reports approximately 6 GPU hours on H100 and \$5 per kernel in their Appendix E.

We further analyze how the API and computation costs relate to CudaForge’s performance, as illustrated in Figure 6a and 6b. The performance of CudaForge monotonically improves as the API and computation time increase. Notably, CudaForge already surpasses the Agentic baseline [?] while using no more than \$0.15 and 10 minutes per task, demonstrating its strong cost–performance tradeoff.

We attribute CudaForge’s cost efficiency to three main factors:

1. **leveraging hardware feedback.** The Judge leverages hardware feedback to diagnose the

current bottleneck and provide targeted, actionable optimization guidance to the Coder. This focused refinement avoids blind exploration and accelerates convergence, thereby reducing the number of API calls and profiling rounds.

2. **Selective NCU metrics.** Instead of profiling the entire metric set, CudaForge uses a curated subset of critical NCU metrics. This not only shortens NCU profiling time but also reduces API cost, as fewer metrics decrease the input token length in Judge queries.
3. **Lightweight memory.** In each iteration, both the Coder and Judge are prompted with current round’s information, instead of using the full conversation history. This lightweight memory design minimizes redundant context tokens and computation overhead, allowing each agent to focus solely on the most recent feedback and candidate kernel.

3.6 Ablation Studies

Comparison with using the entire set of NCU metrics. A key design choice in CudaForge is to filter the full set of NCU metrics and retain only a subset of 24 critical metrics for the Judge. This selective design allows the Judge to focus on the most informative performance indicators, avoiding redundancy and enabling more consistent optimization feedback. We conduct an ablation study to evaluate this choice. As shown in Table 1, using the complete set of NCU metrics leads to lower correctness and performance, as the Judge is overwhelmed by excessive, partially redundant signals. Moreover, profiling with all NCU metrics substantially increases inference cost—each kernel requires approximately 40 minutes on an RTX 6000 GPU and incurs about \$1 in API cost—whereas our selective-metric design achieves superior performance with far lower overhead (26.5 minutes on an RTX 6000 and \$0.3 in API cost). These results confirm that concise, focused hardware feedback is both more effective and more efficient than exhaustive profiling. We also provide a case study in Appendix B.1.

Comparison with o3-self-refine. A key motivation behind CudaForge is to decouple the roles of generation and evaluation. In o3-self-refine, the same model performs ten rounds of self-refinement, implicitly taking on both roles: it must both propose new kernels and evaluate its own outputs based on hardware feedback and runtime signals. While this strategy improves the correctness percentage from 57.6% to 92.8%, performance remains limited ($1.107\times$ speedup, 55.2% Fast₁). In contrast, CudaForge explicitly separates responsibilities: the Coder focuses on code generation, while the Judge specializes in providing structured feedback. This division of labor proves critical—allowing each agent to concentrate on a distinct reasoning process—and results in significantly higher efficiency ($1.677\times$ speedup, 70.8% Fast₁) without sacrificing correctness.

Comparison with o3-correction (correction-only Judge). In o3-correction, the Judge only provides correction feedback based on runtime signals, without optimization feedback. This setting achieves the same 97.6% correctness as CudaForge, confirming that iterative error correction is sufficient to ensure reliable kernel generation. However, efficiency remains much lower, with only $1.222\times$ performance and 58.8% Fast₁. The contrast with CudaForge (**$1.677\times$, 70.8%**) highlights that while correctness feedback stabilizes generation, performance feedback (grounded in hardware profiling) is essential for driving substantial efficiency gains.

Comparison with o3-optimization (optimization-only Judge). We also evaluate the variant where the Judge provides only optimization feedback, without correction feedback. In this setting,

Table 4: CudaForge’s performance on different GPUs. The system consistently achieves high correctness and strong performance across architectures by incorporating GPU specifications and *Nsight Compute* profiling signals during optimization.

GPU	Correct $\uparrow$	Median $\uparrow$	75% $\uparrow$	Perf $\uparrow$	Fast ₁ $\uparrow$
RTX 6000(Ada Arch-Data center level)	100%	1.322	1.736	1.767	84.0%
RTX 4090(Ada Arch-Desktop level)	100%	1.188	1.589	1.327	80.0%
A100(Ampere Arch-Data center level)	100%	1.371	1.762	1.841	84.0%
RTX 3090(Ampere Arch-Desktop level)	100%	1.155	1.706	1.320	72.0%

the Coder frequently generates kernels that fail to compile or run, since functional errors remain uncorrected. As a result, overall correctness is substantially lower than CudaForge, and the potential benefits of optimization guidance cannot be realized. This outcome demonstrates that correctness feedback is a prerequisite: Without first ensuring functional validity, optimization feedback alone is ineffective and often wasted. In contrast, CudaForge leverages both correction and optimization feedback, enabling stable kernel generation and consistent efficiency improvements.

3.7 Generalization Capability of CudaForge

In this section, we analyze CudaForge’s capabilities across various maximum iteration num N , GPU architectures and base models. Considering the high cost of full experiment, we use the stratified subset $\mathcal{D}^*$ for this section.

Scaling up the maximum number of iteration rounds. We investigate the effect of the maximum iteration number N on CudaForge’s performance.

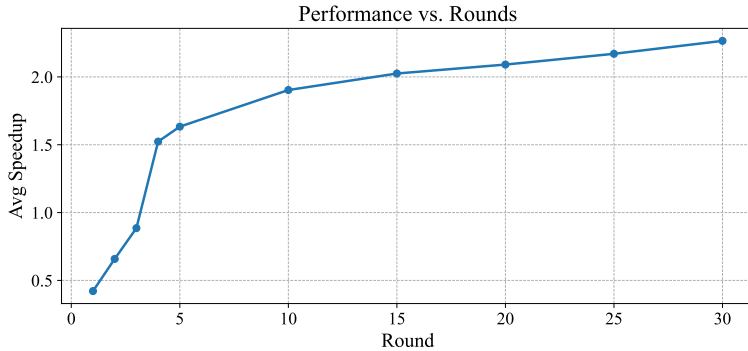


Figure 7: Scaling the number of iteration rounds to 30 on KernelBench (subset $\mathcal{D}^*$).

As shown in Figure 7, increasing N from 1 to 10 leads to substantial performance gains, indicating that CudaForge can rapidly improve kernel efficiency through iterative refinement. Further increasing N from 10 to 30 continues to improve performance, though with a slower growth rate, suggesting that the system gradually approaches its performance ceiling. After 30 rounds of optimization, CudaForge increases the average speedup to $2.271\times$. These results demonstrate that CudaForge benefits from scaling up and has the potential to achieve even stronger performance given larger N with additional inference cost.

Using CudaForge in different GPUs. We also evaluate CudaForge on various GPU architectures, including RTX 6000, RTX 4090, RTX 3090 and A100, to examine its effectiveness under different hardware conditions. As shown in Table 4, CudaForge consistently achieves high correctness and strong performance on all tested GPUs. This is a direct consequence of its design: during the

Table 5: Performance of `CudaForge` with different base model combinations. We fix one agent as OpenAI-o3(denoted as O3) and replace the other with various models. All combinations achieve strong results, showing that the framework is not tied to a specific base model.

Models (Coder/Judge)	Correct $\uparrow$	Median $\uparrow$	75% $\uparrow$	Perf $\uparrow$	Fast ₁ $\uparrow$
O3 / O3	100%	1.322	1.736	1.767	84.0%
O3 / GPT-5	100%	1.131	1.561	2.114	96.0%
O3 / Claude-Sonnet-4	100%	1.265	1.456	1.829	84.0%
O3 / GPT-OSS-120B	100%	1.226	1.490	1.364	76.0%
GPT-5 / O3	100%	1.125	1.388	1.896	72.0%
Claude-Sonnet-4 / O3	88%	1.052	1.207	1.398	56.0%
GPT-OSS-120B / O3	96%	1.080	1.477	1.653	68.0%
QwQ / O3	84%	0.965	1.153	0.790	44.0%

optimization phase, the Judge explicitly incorporates hardware feedback, including NCU metrics and GPU specifications when generating feedback to Coder. This allows the Coder to produce kernels that are tailored to the target GPU at inference time, without training.

Instantiate `CudaForge` with various LLM. To examine whether `CudaForge` depends on a specific base model, we conduct experiments by fixing one side (Coder or Judge) as *OpenAI-o3*(denoted as O3) and replacing the other with various advanced LLMs, including *QwQ-32B*, *GPT-5*, *Claude-Sonnet-4*, and *GPT-OSS-120B*. As shown in Table 5, all combinations achieve high correctness and strong performance, comparable to or even surpassing the original O3/O3 configuration. These results indicate that `CudaForge` is not tied to a specific base model: its effectiveness stems from the workflow of Coder and Judge, and it can readily benefit from stronger models as they emerge.

4 Case study

In this section, we present a case study on a single task to illustrate how the Judge diagnoses issues and recommends optimizations. Figure 8 depicts the 10-round refinement process of `CudaForge` on task `95_CrossEntropyLoss`. We highlight four representative rounds—three optimization rounds and one repair round—to demonstrate how the Judge leverages hardware feedback from NCU to provide targeted optimization or bug-fix suggestions.

In round 2, which is an optimization round, the Judge notices that 23.7% of active warps are stalled due to barrier-type dependencies, which means roughly one quarter of potential issue opportunities are blocked by synchronization. According to this, the Judge recommended replacing the original shared-memory reduction that required multiple block-level synchronizations with a warp-level shuffle reduction, giving below suggestion as prompt for coder: use warp-level shuffles in the max and sum phases, then perform a single cross-warp combine, reducing `__syncthreads()` per block from 16 to 2 (a reduction of 14). After applying this change, performance improved from **1.66 $\times$** to **2.42 $\times$** , with barrier stalls reduced and instruction-issue efficiency increased.

In round 5, it is a correction round. The previous round fails a numerical check with the following error: “Outputs are not close, indicating a result mismatch”. The Judge diagnosed the root cause as an uninitialized target_logit in thread 0 (“Thread-0 uses uninitialized target_logit”), which means the variable target_logit is not updated to thread 0, leading wrong computing results. Accordingly, the Judge gave the minimal fix suggestion, broadcast `target_logit` via `_shfl_sync` to thread 0.

KernelBench Level 1 Task 95: Judge Outputs & Speedup

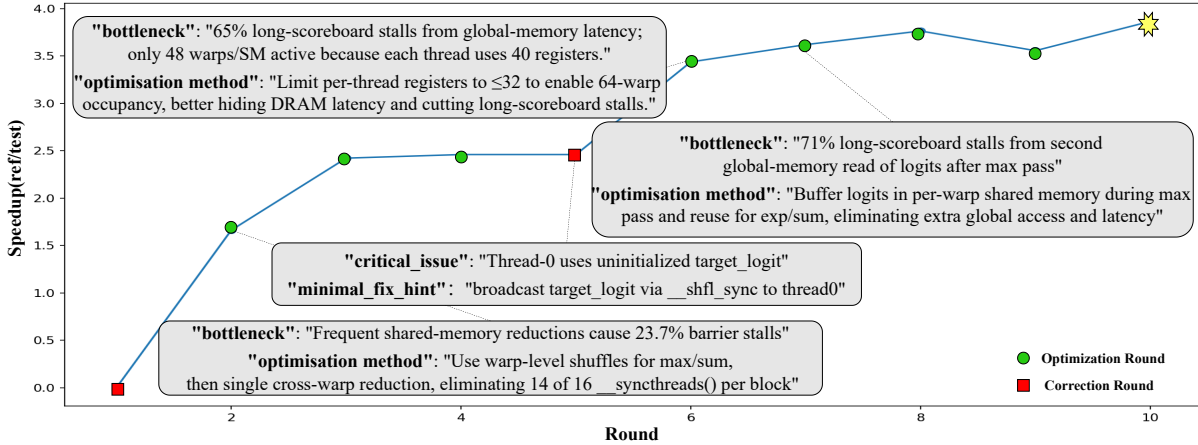


Figure 8: Illustration of the Judge’s outputs—bottleneck diagnoses and optimization suggestions—on KernelBench Level-1 Task 95 (CrossEntropyLoss), as well as the correspondi speedup across rounds (green = optimization, red = correction).

After applying the fix, the numerical issue disappeared.

In rounds 6 & 7 (both optimization rounds), the Judge continues to track `smsp_warp_issue_stalled_long_scoreboard_per_warp_active.pct`. In round 6, this metric is about 65%, primarily reflecting long-scoreboard stalls caused by global-memory latency. Per-thread register usage is high, resulting in limited occupancy (only ~ 48 active warps/SM) and insufficient latency hiding. The recommendation is to reduce per-thread registers to raise concurrency to ~ 64 warps/SM and thereby lower the long-scoreboard share. In round 7, the metric rises to about 71%, rooted in a second global read of logits after the max pass. The Judge therefore advises buffering logits in per-warp shared memory during the max pass and reusing them in the expsum phase, eliminating the redundant global memory access. Together, these strategies reduce global memory access, significantly cut long-scoreboard stalls, improve issue efficiency and throughput; after these two rounds, the speedup increases from $3.436\times$ to $3.762\times$.

This task demonstrates our CudaForge’s stability and expert-like workflow: first analyzing bottlenecks from hardware feedback, then deriving the corresponding optimization strategy.

5 Supplement Observations

Observations in CUDA-L1 results. We carefully examined the kernel outputs reported by CUDA-L1 (see Appendix C) and identified an interesting phenomenon that we term *"fake kernels"*. These kernels, while reported as performant, often contain no actual CUDA code. Instead, they rely on `try-except` constructs and fall back to PyTorch’s official implementations to solve the task. This observation highlights a fundamental challenge in evaluating LLM-generated CUDA kernels. To avoid this issue, we have manually checked all kernels in our experiments.

6 Conclusion

We presented **CudaForge**, a training-free multi-agent framework for CUDA kernel generation and optimization. The framework imitates the iterative workflow of human experts, explicitly incorporating hardware feedback to guide targeted kernel refinement rather than blind exploration. On the KernelBench benchmark, **CudaForge** achieves highest correctness rate and significant performance gains compared with all existing method, while also demonstrating robustness across diverse GPU architectures and base LLMs. Moreover, its performance scales effectively with the number of refinement rounds. Finally, thanks to its low API and time cost, **CudaForge** provides a practical and efficient solution for automated CUDA kernel development.

A Description of the Prompts used in CudaForge Workflow

A.1 Prompt for Coder in first round of generation

We adopt the *One-shot Baseline Prompt* introduced in KERNELBENCH as our initial prompt for first round generation of all the baselines and our method. The full prompt is shown below.

Prompt for Coder in first round of generation

You write custom CUDA kernels to replace the PyTorch operators in the given architecture to get speedups. You have complete freedom to choose the set of operators you want to replace. You may decide to replace some operators with custom CUDA kernels and leave others unchanged. You may replace multiple operators with custom implementations, consider operator fusion opportunities (combining multiple operators into a single kernel, for example, combining `matmul+relu`), or algorithmic changes (such as online softmax). You are only limited by your imagination. Here is an example to show you the syntax of inline-embedding custom CUDA operators in PyTorch.

The example given architecture is:

```
{few_base}
```

The example new architecture with custom CUDA kernels looks like this:

```
{few_new}
```

You are given the following architecture:

```
{arch_src}
```

Optimize the architecture named `Model` with custom CUDA operators! Name your optimized output architecture `ModelNew`. Output the new code in code blocks. Please generate real code, *NOT* pseudocode. Make sure the code compiles and is fully functional. Just output the new model code, no other text, and **NO** testing code!

A.2 Prompt for Judge

In our prompt design for the Judge agent, we place the role specification and output schema in the system prompt. Inspired by [? ? ?], we design the prompts for correction and optimization, which are shown below.

Prompt for CUDA Kernel Correction

You are a senior CUDA + PyTorch correctness auditor. Your job is to read a PyTorch reference and a CUDA candidate and report **exactly one** most critical correctness issue in the CUDA code that would cause a behavioral mismatch vs. the PyTorch reference. Be terse and precise.

Rules:

- Return **one and only one** issue – the single highest-impact problem.
- Prefer semantic/correctness issues over micro-optimizations or style.
- If multiple issues exist, pick the one that most changes outputs or gradients.

- If nothing clearly wrong is found, say it explicitly.
- Keep each field brief; avoid extra commentary, lists, or alternatives.

Output format (JSON):

```
{
  "critical_issue": "<max 20 words>",
  "why_it_matters": "<max 35 words>",
  "minimal_fix_hint": "<max 20 words>"
}
```

You are given:

ERROR_LOG:

{ERROR_LOG}

PyTorch reference (ground truth):

{PYTORCH_CODE}

CUDA candidate (to audit):

{CUDA_CODE}

Follow the Rules and produce the JSON exactly in the specified format.

Prompt for CUDA Kernel Optimization

You are a senior CUDA performance engineer. Read the target GPU spec, the PyTorch reference code, the current CUDA candidate, and the Nsight Compute metrics. Then identify **exactly one** highest-impact speed bottleneck by 3-4 most important metrics, propose **exactly one** optimisation method and propose a modification plan. Be surgical and metrics-driven.

Rules:

- Return **one and only one** optimisation method – the largest expected speedup.
- Prefer changes that directly address measured bottlenecks (occupancy limits, memory coalescing, smem bank conflicts, register pressure, long/short scoreboard stalls, tensor-core underutilisation, etc.).
- Keep fields brief; avoid lists of alternatives, disclaimers, or generic advice.

Output format (JSON):

```
{
  "bottleneck": "<max 30 words>",
  "optimisation method": "<max 35 words>",
  "modification plan": "<max 35 words>"
}
```

Target GPU

GPU Name: {gpu_name}
Architecture: {gpu_arch}
Details:
{gpu_items}

PyTorch Reference

{python_code}

CUDA Candidate

{CUDA_CODE}

Nsight Compute metrics (verbatim)

{NCU_METRICS}

Read everything and follow the Rules exactly. Return the JSON in the specified format.

A.3 Prompt for Coder from Round 2 to N

After getting the feedback from the Judge, the Coder then corrects or optimizes the current kernel candidate based on the feedback. Prompts for correction and optimization are shown below.

Prompt for Kernel Correction

You are a senior CUDA-extension developer. Your job is to **FIX** the compilation or runtime errors in the Python script shown below.

OUTPUT RULES (STRICT)

- Inside the block, follow exactly this order:
 - Imports - torch, torch.nn, load_inline.
 - source - triple-quoted CUDA string(s) (kernel + host wrapper).
 - cpp_src - prototypes for all kernels you expose.
 - One load_inline call per kernel group.
 - class ModelNew(nn.Module) - mirrors original inputs/outputs but calls your CUDA kernels.
- Do NOT include testing code, if `__name__ == "__main__"`, or extra prose.

ERROR LOG

{ERROR_LOG}

OLD CODE (read-only)

{CUDA_CODE}

Main Critical Problem

{Problem}

Output Section (to be generated):

<your corrected code>

Prompt for Kernel Optimization

Target GPU

GPU Name: {gpu_name}

Architecture: {gpu_arch}

Details:

{gpu_items}

You are a CUDA-kernel optimization specialist.

Analyze the provided architecture and **strictly apply the following STRATEGY** to produce an improved CUDA kernel.

{CUDA_CODE}

Optimization instructions:

{optimization_suggestion}

GOAL

- Improve latency and throughput on the target GPU.
- Maintain correctness within atol=1e-4 or rtol=1e-4.
- Preserve the public Python API (same inputs/outputs, shapes, dtypes).

OUTPUT RULES (STRICT)

1. Inside the block, follow exactly this order:
 1. Imports - torch, torch.nn, load_inline.
 2. source - triple-quoted CUDA string(s) (kernel + host wrapper).
 3. cpp_src - prototypes for all kernels you expose.
 4. One load_inline call per kernel group.
 5. class ModelNew(nn.Module) - mirrors original inputs/outputs but calls your CUDA kernels.
2. Do NOT include testing code, if __name__ == "__main__", or extra prose.

Output Section (to be generated):

<your corrected code>

B Detail for the NCU metrics

B.1 Why choose NCU subset metrics?

We find that exposing large models to the full NCU metric set overwhelms them, reducing the accuracy and stability of their optimization suggestions and degrading Judge output quality. We illustrate this with following specific case study.

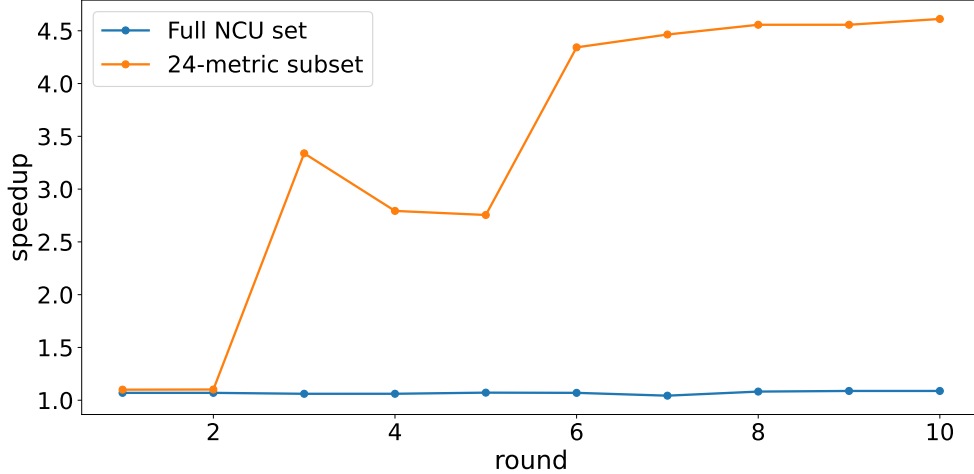


Figure 9: Generated kernel performance comparison between Full-metrics input(blue line) vs. Metrics subset input(orange line) for Judge agent on Level 2 task 51.

As shown in Figure 9, we compare two configurations on a representative task. (i) The Judge agent receives *all* raw NCU metrics in each round and provides feedback to the Coder agent. (ii) The Judge agent receives only the 24 key metrics selected by our method and captures only 3-4 critical metrics in each round to generate optimization suggestion.

Within 10 iterative rounds, the Judge agent using full NCU metrics fails to guide the Coder agent toward better kernels. The performance of the generated kernels remains almost unchanged, despite repeated interactions. In contrast, when the Judge agent is restricted to use subset metrics, the performance of generated kernels varies across rounds and is often improved. This process eventually leads to a substantial speedup.

In addition, we carefully reviewed how the Judge identified bottlenecks and proposed optimizations in both configurations, full set vs. 24-subset. We select one Judge’s response from the optimization round to better illustrate the difference in judge output, with full NCU and with a subset of NCU as judge input.

Response by using the full NCU metrics set

```
{"bottleneck": "Long-scoreboard stalls from re-loading GEMM output out of DRAM (gemm_out round-trip dominates cycles).",
```

```
"optimisation method": "Fuse the Linear GEMM and the current epilogue into one FP16 tensor-core CUTLASS kernel that performs subtract, row-mean, GELU and residual add in-register, avoiding global-memory re-reads.",
```

```
"modification plan": "Write CUTLASS custom epilogue functor; use cublasLt or cutlass::
```

```
gemm_universal to compute GEMM+A, keep accumulators in registers, warp-reduce to mean,
apply GELU, add x_orig, directly store final output; expose via new PyTorch extension."}]
```

Response by using the 24 key NCU subset

```
{"bottleneck": "Kernel is DRAM-bound (102.9 % peak); 57 % long-scoreboard stalls from
second pass reading original_x.",
```

```
"optimisation method": "Fuse the second loop: cache each thread's original_x elements
in registers during diff accumulation, then add gelu_val and store, eliminating the
extra B×F global read.",
```

```
"modification plan": "Replace two for-loops with one: load linear_out, subtract, and
original_x together, keep original_x[16] in per-thread array, finish reduction, add
gelu_val, write to output; ~4 MB traffic per batch, >30 % speedup."}]
```

Based on these two responses, we find that judge with full set NCU metrics tends to misidentify the true bottleneck. The judge with full set NCU metrics attributes the bottleneck to re-loading `gemm_out` and recommends a monolithic CUTLASS epilogue that performs row-mean/GELU/residual in registers. This diagnosis is misaligned with our kernel's access pattern and is hard to realize for general shapes due to cross-tile aggregation. In contrast, the judge with 24-key subset correctly identifies a DRAM-bound kernel dominated by the second pass over `x_orig`, and proposes a one-pass rewrite that caches `x_orig` in registers during the first traversal and writes back after GELU, eliminating an entire $B \times F$ global memory read. This change is lightweight, architecture-agnostic, and yields consistent speedups (e.g., about 4 MB less traffic per batch, more than 30% in our setting).

B.2 Top-20 NCU metrics Example

This section reports, in several example tasks, the Top-20 Nsight Compute (NCU) metrics most correlated with runtime, ranked by the absolute value of the Pearson correlation coefficient. Here, runtime refers to the kernel's execution time. When the correlation coefficient is positive, larger metric values typically imply longer execution time; when it is negative, larger metric values typically imply shorter execution time. All metric names follow their original name in NCU.

Table 6: Task-Conv2D: Pearson correlation with runtime (Top-20).

Metric Name	Correlation	Abs Correlation
<code>sm__cycles_active.avg</code>	1.000 000	1.000 000
<code>gpc__cycles_elapsed.max</code>	1.000 000	1.000 000
<code>launch__occupancy_limit_shared_mem</code>	0.945 507	0.945 507
<code>dram__bytes.sum.per.second</code>	-0.924 251	0.924 251
<code>gpu__dram_throughput.avg.pct.of.peak.sustained.elapsed</code>	-0.924 155	0.924 155
<code>smsp__inst_executed.avg</code>	0.916 287	0.916 287
<code>smsp__inst_executed.sum</code>	0.916 287	0.916 287
<code>smsp__inst_issued.avg</code>	0.916 262	0.916 262
<code>smsp__inst_issued.sum</code>	0.916 262	0.916 262
<code>lts__t_sector_hit_rate.pct</code>	0.839 237	0.839 237
<code>smsp__sass_average_branch_targets_threads_uniform.pct</code>	0.810 334	0.810 334
<code>lts__throughput.avg.pct.of.peak.sustained.elapsed</code>	-0.787 261	0.787 261
<code>smsp__inst_executed_op_branch.sum</code>	0.746 483	0.746 483
<code>launch__grid_size</code>	0.745 917	0.745 917
<code>l1tex__t_sector_hit_rate.pct</code>	0.728 356	0.728 356
<code>gpc__cycles_elapsed.avg.per.second</code>	0.728 053	0.728 053

Continued on next page

Metric Name	Correlation	Abs Correlation
dram__cycles_elapsed.avg.per_second	0.665 784	0.665 784
launch__waves_per_multiprocessor	0.627 478	0.627 478
launch__thread_count	0.627 478	0.627 478
launch__shared_mem_per_block_static	−0.610 501	0.610 501

Table 7: Task-SpMM: Pearson correlation with runtime (Top-20).

Metric Name	Correlation	Abs Correlation
gpc__cycles_elapsed.max	0.999 993	0.999 993
sm__cycles_active.avg	0.998 432	0.998 432
gpu__compute_memory_request_throughput.avg.pct...	−0.967 284	0.967 284
gpu__compute_memory_throughput.avg.pct_of_peak...	−0.964 455	0.964 455
lts__t_sector_hit_rate.pct	0.951 201	0.951 201
dram__bytes.sum.per_second	−0.926 134	0.926 134
gpu__dram_throughput.avg.pct_of_peak_sustained...	−0.925 856	0.925 856
l1tex__throughput.avg.pct_of_peak_sustained_active	0.871 262	0.871 262
sm__inst_executed.avg.per_cycle_elapsed	−0.837 675	0.837 675
smsp__issue_inst0.avg.pct_of_peak_sustained_active	0.837 284	0.837 284
smsp__issue_active.avg.pct_of_peak_sustained...	−0.837 284	0.837 284
smsp__issue_active.avg.per_cycle_active	−0.837 283	0.837 283
sm__inst_issued.avg.per_cycle_active	−0.836 185	0.836 185
sm__inst_issued.avg.pct_of_peak_sustained_active	−0.836 185	0.836 185
sm__inst_executed.avg.per_cycle_active	−0.836 160	0.836 160
sm__instruction_throughput.avg.pct_of_peak_sust...	−0.806 478	0.806 478
smsp__average_warp_latency_per_inst_issued.ratio	0.802 793	0.802 793
smsp__average_warps_active_per_inst_executed.ratio	0.802 777	0.802 777
derived__smsp__inst_executed_op_branch_pct	−0.728 768	0.728 768
smsp__warps_eligible.avg.per_cycle_active	−0.630 772	0.630 772

B.3 Key Subset of 24 NCU Metrics

The table below lists the exact 24 metrics in our NCU key subset, as a result of Algorithm 1, 2

Table 8: The 24-metric key subset.

#	Metric Name
1	sm__cycles_active.avg
2	sm__warps_active.avg.pct_of_peak_sustained_active
3	launch__occupancy_limit_blocks
4	launch__occupancy_limit_registers
5	launch__occupancy_limit_shared_mem
6	launch__registers_per_thread
7	sm__inst_executed.sum
8	sm__inst_executed_pipe_fp32.avg.pct_of_peak_sustained_active
9	sm__inst_executed_pipe_tensor.avg.pct_of_peak_sustained_active
10	dram__bytes_read.sum
11	dram__bytes_write.sum
12	dram__throughput.avg.pct_of_peak_sustained_elapsed
13	dram__bytes.sum.per_second
14	gpu__dram_throughput.avg.pct_of_peak_sustained_elapsed
15	l1tex__t_sector_hit_rate.pct
16	l1tex__throughput.avg.pct_of_peak_sustained_active
17	lts__t_sector_hit_rate.pct
18	lts__throughput.avg.pct_of_peak_sustained_active
19	smsp__warp_issue_stalled_memory_dependency_per_warp_active.pct
20	smsp__warp_issue_stalled_short_scoreboard_per_warp_active.pct
21	smsp__warp_issue_stalled_long_scoreboard_per_warp_active.pct
22	smsp__warp_issue_stalled_barrier_per_warp_active.pct

Continued on next page

#	Metric Name
23	smsp__warp_issue_stalled_branch_resolving_per_warp_active.pct
24	smsp__sass_average_branch_targets_threads_uniform.pct

C CUDA-L1

In our replication efforts, we found that the authors of CUDA-L1 released only the final, generated kernels for each task. After carefully studying these cases, we identified several interesting findings.

First, We found that CUDA-L1 tends to emphasize PyTorch-level optimizations rather than generating and refining custom CUDA kernels. This pattern also emerged as the most frequent issue in their provided case. Although CUDA-L1 reports the top-10 cases with the largest speedups, our review shows that nine of these ten final solutions do not use custom CUDA kernels; instead, they rely heavily on official PyTorch implementations. We show several cases of their results below.

This is the top-ranked entry in their *KernelBench Tasks Ranked by RL-CUDA1 Acceleration (Top-10)*: Level 2 Task 83, with a reported 120.3× speedup. It contains no CUDA kernel.

Level 2 Task 83 — Reported 120.3× Speedup

```
import torch
import torch.nn as nn

class ModelNew(nn.Module):
    """
    Optimized implementation of a model that performs a 3D convolution,
    applies Group Normalization, minimum, clamp, and dropout.

    Args:
        in_channels (int): Number of input channels
        out_channels (int): Number of output channels
        kernel_size (int or tuple): Size of the convolving kernel
        groups (int): Number of groups for GroupNorm
        min_value (float): Minimum value for clamp operation
        max_value (float): Maximum value for clamp operation
        dropout_p (float): Dropout probability
    """
    def __init__(self, in_channels, out_channels, kernel_size, groups, min_value,
max_value, dropout_p):
        super(ModelNew, self).__init__()
        # Store the original layers for parameter compatibility
        self.conv = nn.Conv3d(in_channels, out_channels, kernel_size)
        self.norm = nn.GroupNorm(groups, out_channels)
        self.dropout = nn.Dropout(dropout_p)
        self.min_value = min_value
        self.max_value = max_value
        self.dropout_p = dropout_p

        # Pre-compute convolution parameters for output shape calculation
```

```

        if isinstance(kernel_size, int):
            self.kernel_size = (kernel_size, kernel_size, kernel_size)
        else:
            self.kernel_size = kernel_size

        self.stride = self.conv.stride
        self.padding = self.conv.padding
        self.dilation = self.conv.dilation

    def forward(self, x):
        x = self.conv(x)
        x = self.norm(x)
        x = torch.minimum(x, torch.tensor(self.min_value, device=x.device))
        x = torch.clamp(x, min=self.min_value, max=self.max_value)
        x = self.dropout(x)
        return x

# Keep ALL hyperparameters EXACTLY as shown in the reference implementation
batch_size = 128
in_channels = 3
out_channels = 16
depth, height, width = 16, 32, 32
kernel_size = 3
groups = 8
min_value = 0.0
max_value = 1.0
dropout_p = 0.2

def get_inputs():
    return [torch.randn(batch_size, in_channels, depth, height, width)]

def get_init_inputs():
    return [in_channels, out_channels, kernel_size, groups, min_value, max_value,
            dropout_p]

```

The second-ranked case is Level-1 Task 12 (Matmul with diagonal matrices), with a reported 64.4× speedup, which also contains no CUDA kernel:

Level 1 Task 12 — Reported 64.4× Speedup

```

# diag_mm_compare.py
import time
import math
import torch
import torch.nn as nn
import torch.nn.functional as F

# -----

```

```

# Reference implementation
# -----
class Model(nn.Module):
    """
    Simple model that performs a matrix multiplication of a diagonal matrix with another
    matrix.
    C = diag(A) * B
    """
    def __init__(self):
        super(Model, self).__init__()

    def forward(self, A, B):
        """
        Args:
            A (torch.Tensor): 1D tensor, diagonal entries. Shape: (N,)
            B (torch.Tensor): 2D tensor. Shape: (N, M)
        Returns:
            torch.Tensor: (N, M)
        """
        return torch.diag(A) @ B

# -----
# Optimized implementation
# -----
class ModelNew(nn.Module):
    """
    Optimized model that performs a matrix multiplication of a diagonal matrix with another
    matrix.
    C = diag(A) * B
    """
    def __init__(self):
        super(ModelNew, self).__init__()

    def forward(self, A, B):
        """
        Args:
            A (torch.Tensor): 1D tensor, diagonal entries. Shape: (N,)
            B (torch.Tensor): 2D tensor. Shape: (N, M)
        Returns:
            torch.Tensor: (N, M)
        """
        # Equivalent to torch.diag(A) @ B, but avoids forming the full diagonal matrix
        return B * A.unsqueeze(1)

# -----

```

```

# Hyperparameters & inputs
# -----
M = 4096
N = 4096

def get_inputs(device=None, dtype=torch.float32):
    A = torch.randn(N, device=device, dtype=dtype)
    B = torch.randn(N, M, device=device, dtype=dtype)
    return [A, B]

def get_init_inputs():
    return [] # No special initialization inputs needed

```

In addition, we observed many reported speedups that are effectively equal to one (clustered around 1.00, typically within $\pm 5\%$). A closer inspection shows that, in these cases, the system falls back to the original PyTorch operator when the custom kernel fails to compile, which naturally yields no measurable speedup.

For example, below is the forward method from the final solution for KernelBench Level-1 Task 3 generated by CUDA-L1. This code get from the CUDA-L1's official Github. We observe that the method first attempts to call a *custom CUDA kernel*; however, upon any compilation failure or exception, it immediately falls back to `torch.bmm(A, B)`. Crucially, `torch.bmm(A, B)` is exactly the operator that this task asks to be replaced by a custom kernel, meaning the fallback undermines the task's objective. This explains why the reported speedup is only $1.006\times$.

Level 1 Task 3 — Reported $1.006\times$ Speedup

```

def forward(self, A: torch.Tensor, B: torch.Tensor) -> torch.Tensor:
    """
    Performs batched matrix multiplication.

    Args:
        A: Input tensor of shape (batch_size, m, k).
        B: Input tensor of shape (batch_size, k, n).

    Returns:
        C: Output tensor of shape (batch_size, m, n).
    """
    # Fall back to torch.bmm if CUDA module failed to load
    if ModelNew._cuda_module is None:
        return torch.bmm(A, B)

    # Check if inputs are on CUDA
    if not A.is_cuda or not B.is_cuda:
        A = A.cuda() if not A.is_cuda else A
        B = B.cuda() if not B.is_cuda else B

    # Ensure inputs are contiguous and float32
    A = A.contiguous().float()

```

```

B = B.contiguous().float()

# Use custom CUDA kernel
try:
    result = ModelNew._cuda_module.batched_matmul(A, B)
    if not A.is_cuda:
        result = result.cpu()
    return result
except Exception as e:
    print(f"Error in custom kernel: {e}, falling back to torch.bmm")
    return torch.bmm(A, B)

```

D Details of Benchmark

D.1 KernelBench

KernelBench is a standardized benchmark designed to evaluate the capability of large language models (LLMs) in CUDA kernel generation and optimization. It consists of 270 tasks across four levels of increasing difficulty, of which Levels 1–3 (250 tasks in total) are commonly adopted for evaluation. Each task provides a PyTorch reference implementation together with fixed input–output specifications, enabling automated correctness and performance validation.

- **Level 1 (Basic Operators):** Contains simple, low-level operators such as matrix multiplication, element-wise operations, and reductions. These tasks primarily test the ability to generate functionally correct CUDA kernels.
- **Level 2 (Composite Operations):** Involves multi-step operator combinations, requiring the model to compose multiple CUDA primitives and manage intermediate memory efficiently. These tasks test the capacity for more complex code synthesis.
- **Level 3 (End-to-End Models):** Includes challenging kernels derived from full neural network architectures such as AlexNet, VGG, and ResNet components. These tasks assess the ability to produce efficient, large-scale kernels under realistic deep learning workloads.
- **Level 4 (Optional):** The full benchmark also defines an advanced level with additional research-oriented tasks, but this is less frequently adopted due to its complexity and lack of standardized evaluation setups.

KernelBench has become a widely used benchmark in recent work on LLM-based code generation [? ? ?], as it provides a controlled and reproducible environment to measure both *correctness* (functional equivalence to PyTorch) and *efficiency* (execution speed relative to PyTorch). In our study, we adopt all Level 1–3 tasks, following prior work, to ensure fair comparison across baselines.

D.2 Our stratified random subset $\mathcal{D}^*$

While our main evaluation is conducted on the full KernelBench Level 1–3 benchmark (250 tasks in total), we additionally construct a stratified subset $\mathcal{D}^*$ to enable detailed analysis and fair comparison, while reducing computation cost in experiments.

The construction of $\mathcal{D}^*$ follows two principles: (1) **Coverage across difficulty levels.** Since KernelBench is stratified by increasing task complexity (Level 1: single-operator tasks, Level 2: multi-step fused operators, Level 3: full network components), we ensure that the sampled subset preserves the relative distribution of difficulty. (2) **Diversity of task types.** Within each level, we sample tasks uniformly across different operator categories (e.g., elementwise ops, reductions, convolutions, fused blocks) so that the subset remains representative of the overall benchmark.

Concretely, we perform stratified random sampling with a fixed 10% ratio for each level, resulting in a subset of 10 tasks from Level 1, 10 tasks from Level 2, and 5 tasks from Level 3, for a total of 25 tasks. For reproducibility, the exact task IDs included in $\mathcal{D}^*$ are:

- **Level 1 (10 tasks):** 13, 10, 16, 29, 35, 72, 7, 89, 93, 34
- **Level 2 (10 tasks):** 17, 19, 40, 3, 13, 21, 38, 28, 26, 34
- **Level 3 (5 tasks):** 5, 18, 32, 41, 21